NLP

Complete Developer Study Notes

Concepts • Practical Examples • Projects • Interview Preparation

Prepared by: Rahul Yadav

20 Pages

Rahul Yadav

1. Introduction to NLP

1.1 What is NLP ?

- NLP (Natural Language Processing) is a branch of AI that helps computers understand, process and generate human language.
- It combines linguistics, machine learning and deep learning techniques.
- NLP enables machines to work with text and speech data in a meaningful way.

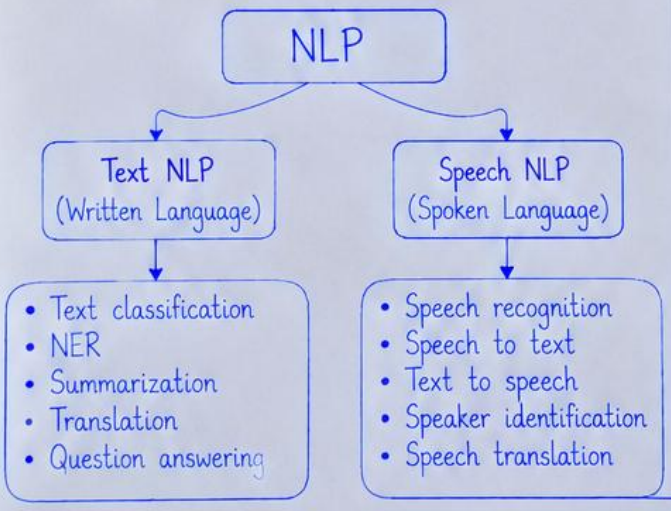
1.2 Examples of NLP Tasks

- Text classification (e.g. spam detection)
- Sentiment analysis (positive / negative)
- Named Entity Recognition (NER)
- Text summarization
- Machine translation (e.g. Hindi → English)
- Question answering (Q&A)
- Chatbots and virtual assistants

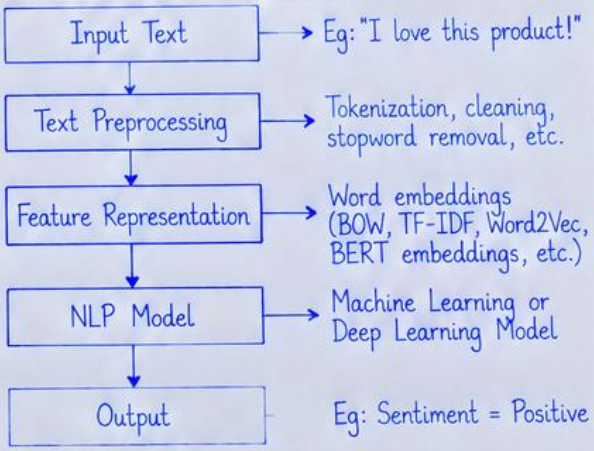
1.3 Applications of NLP

- Search engines (better search results)
- Social media (sentiment analysis)
- Customer support chatbots
- Machine translation (Google Translate)
- Summarizing long documents
- Healthcare (medical text analysis)
- Finance (news and report analysis)
- Education (automated grading, tutoring)
- Voice assistants (Siri, Alexa, Google Assistant)

1.4 Types of NLP



1.5 How NLP Works ?



1.6 Common NLP Models (Evolution)

Model	Year	Description
Bag of Words (BoW)	1990s	Simple word count based representation
TF-IDF	1990s	Weighted word representation
Word2Vec	2013	Dense word embeddings
GloVe	2014	Global word co-occurrence embeddings
RNN / LSTM	2010s	Sequence models for text
Transformer	2017	Attention based model
BERT	2018	Pre-trained language model (for understanding text)
GPT	2018	Generative language model (for text generation)

1.7 Challenges in NLP

- Understanding context and meaning
- Handling ambiguity (same word, multiple meanings)
- Sarcasm and humor detection
- Different languages and dialects
- Large vocabulary and rare words
- Domain specific terminology
- Need for large and high quality data

1.8 Future of NLP

- More accurate and human like language understanding
- Multimodal NLP (text + image + audio)
- Real time and personalized AI assistants
- Better support for low resource languages
- Responsible and ethical use of NLP

Rahul Yadav

2. Text Preprocessing in NLP

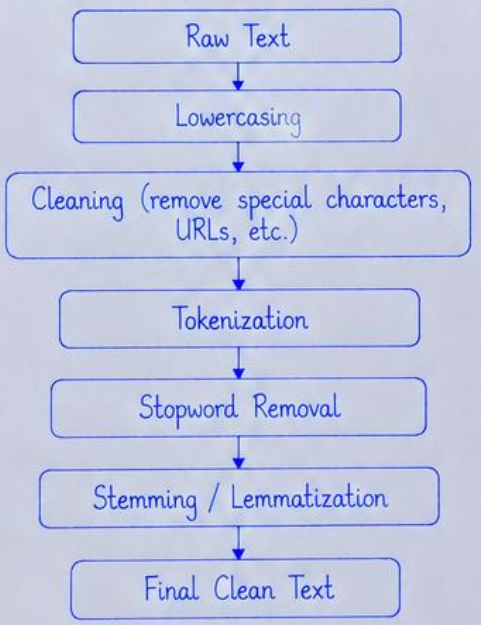
2.1 What is Text Preprocessing ?

- Text preprocessing is the process of cleaning and transforming raw text into a format that can be easily understood by machine learning models.
- It helps in removing noise, standardizing the text and reducing the size of vocabulary.
- It is an essential step in most NLP tasks like classification, sentiment analysis, etc.

2.2 Why is it Important ?

- Improves model performance.
- Reduces unwanted noise and inconsistencies.
- Creates a clean and meaningful representation of text.
- Helps in handling real-world unstructured data.

2.3 Common Steps in Text Preprocessing



2.4 Example

Raw text : "I loved this product!!! It's amazing :) Visit https://example.com"

After lowercasing : "i loved this product!!! it's amazing :) visit https://example.com"

After cleaning : "i loved this product its amazing visit"

After tokenization : [' i', 'loved', 'this', 'product', 'i ts', 'amazing', 'visit"]

After stopword removal : ['loved', 'product', 'amazing', 'visit"]

After stemming : ['love', 'product', 'amaz', 'visit"]

After lemmatization : ['love', 'product', 'amazing', 'visit"]

2.5 Lowercasing

- Converts all characters to lowercase to maintain consistency.
- Example: "Hello World" → "hello world"

2.6 Removing Special Characters

- Removes punctuation, numbers, emojis, URLs, etc.
- Example: "Hi!!! @# AI ☺" → "Hi AI"

2.7 Tokenization

- Splits text into smaller units called tokens (words, subwords or sentences).
- Example: "I love NLP" → ['I', 'love', 'NLP"]
- Types: Word Tokenization, Sentence Tokenization, Subword Tokenization (BPE, WordPiece)

2.8 Stopword Removal

- Removes common words that do not add much meaning (e.g. is, the, and, etc.).
- Example: "this is a good product" → "good product"

2.9 Stemming vs Lemmatization

Aspect	Stemming	Lemmatization
Output	Root word (may not be a valid word)	Meaningful base word (valid word)
Method	Rule based (e.g. Porter Stemmer)	Uses vocabulary and POS tagging
Example	running → runn	running → run
Speed	Faster	Slower
Accuracy	Less accurate	More accurate

2.10 Handling Common Real-World Issues

Issue	Solution
HTML tags	Remove using regex or libraries (BeautifulSoup)
URLs	Remove or replace with <URL> token
Emojis	Remove or convert to text
Numbers	Remove or replace with <NUM>
Extra spaces	Replace multiple spaces with single space
Contractions	Expand (e.g. "don't" → "do not")
Spelling mistakes	Use spell correction libraries (TextBlob)
Mixed case	Convert to lowercase

2.11 Python Example (Using NLTK)

```
import nltk
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer

text = "I loved this product! It's amazing :)"
text = text.lower()
text = ''.join([c for c in text if c.isalnum() or c.isspace()])
tokens = text.split()
stop_words = set(stopwords.words('english'))
tokens = [w for w in tokens if w not in stop_words]
stemmer = PorterStemmer()
final_tokens = [stemmer.stem(w) for w in tokens]
print(final_tokens)
```

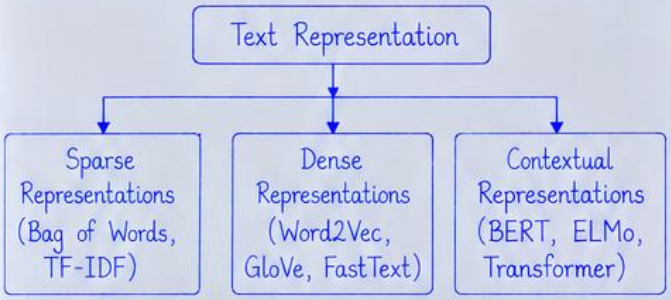
Rahul Yadav

3. Text Representation Techniques

3.1 Why Convert Text to Numbers ?

- Machine learning models can only understand numerical inputs.
- Text needs to be converted into vector representations.
- These numerical representations capture the meaning, context and semantic information of words or sentences.

3.2 Types of Text Representations



3.3 1. Bag of Words (BoW)

- Represents text as the count of words in a document.
- Simple and effective for basic tasks.
- Ignores word order and context.

Example :

Sentence : "I love NLP and NLP is fun"
Vocabulary : [I, love, NLP, and, is, fun]

Word	Count
I	1
love	1
NLP	2
and	1
is	1
fun	1

3.4 2. TF-IDF (Term Frequency - Inverse Document Frequency)

- Gives higher weight to important words and lower weight to common words.
- It is a sparse representation.
- Useful for text classification and search.

Formula :

$$\text{TF-IDF}(w) = \text{TF}(w,d) \times \log\left(\frac{N}{DF(w)}\right)$$

- $TF(w,d)$: Frequency of word w in document d
- $DF(w)$: Number of documents containing word w
- N : Total number of documents

3.5 Example of TF-IDF

Document 1 : "I love machine learning"
Document 2 : "I love deep learning"

Word	TF (D1)	TF (D2)	DF	IDF	TF-IDF (D1)
I	1	1	2	0	0
love	1	1	2	0	0
machine	1	0	1	0.69	0.69
learning	1	1	2	0	0
deep	0	1	1	0.69	0

Note: IDF = $\log(N/DF)$, here $N = 2$ documents.

3.6 3. Word Embeddings (Dense Representation)

- Represent words as dense vectors in a continuous vector space.
- Similar words have similar vector representations.
- Capture semantic meaning and relationships.

Popular Methods :

- Word2Vec (Skip-gram, CBOW)
- GloVe (Global co-occurrence matrix)
- FastText (handles subword information)

3.7 Example: Word Embeddings

Word	Dimension 1	Dimension 2	Dimension 3
king	0.52	0.12	0.87
queen	0.51	0.11	0.86
man	0.43	0.21	0.78
woman	0.42	0.20	0.77

Words with similar meaning have closer vectors.

3.8 4. Contextual Embeddings

- Traditional embeddings give same vector for a word (e.g. "bank").
- Contextual embeddings generate different vectors based on the context.
- Examples: BERT, ELMo, GPT, RoBERTa.

3.9 Example: Contextual Embeddings

Word: "bank"

- Sentence 1: "I went to the bank to deposit money."
  → Represents financial institution
- Sentence 2: "We sat on the bank of the river."
  → Represents river side

BERT generates different vector representations for "bank" in these two contexts.

3.10 Comparison of Text Representation Techniques

Method	Representation	Captures Context	Pros	Cons
BoW	Sparse	No	Simple	Ignores order
TF-IDF	Sparse	No	Better than BoW	Still ignores context
Word2Vec	Dense	Limited	Captures semantics	Same vector for all contexts
GloVe	Dense	Limited	Good performance	No context
FastText	Dense	Limited	Handles subwords	No context
BERT	Dense	Yes	Contextual meaning	Computationally expensive

Rahul Yadav

4. Word Embeddings

4.1 What are Word Embeddings ?

- Word embeddings are dense vector representations of words in a continuous vector space.
- They capture the semantic meaning, context and relationships between words.
- Similar words have similar vector representations.
- They are learned from large text corpora using machine learning models.

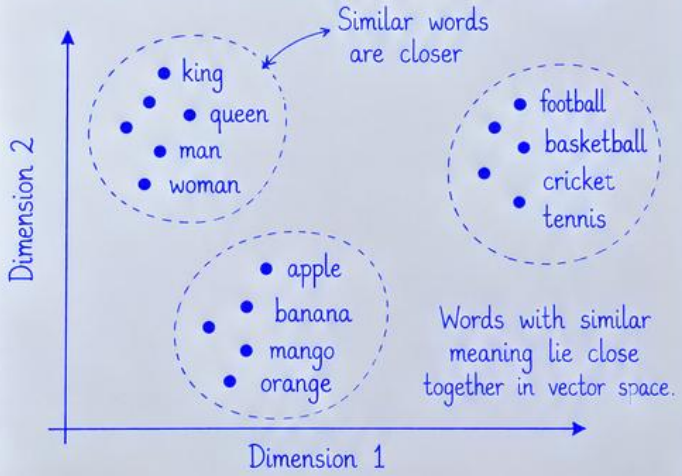
4.2 Why are Word Embeddings Important ?

- Convert words into numerical form that machine learning models can understand.
- Capture semantic meaning and context.
- Help in improving performance in NLP tasks like classification, translation, etc.
- Reduce dimensionality compared to one-hot encoding.
- Enable models to understand similarity and analogies between words.

4.3 One-Hot Encoding vs Word Embeddings

Aspect	One-Hot Encoding	Word Embeddings
Representation	Sparse vector	Dense vector
Vector size	Vocabulary size (e.g. 10,000)	Fixed size (e.g. 100, 300)
Captures meaning	No	Yes
Similar words	No relationship	Similar vectors
Memory usage	High	Low
Example	[0,0,0,1,0,....]	[0.25, -0.12, 0.87, ...]

4.4 Visualization of Word Embeddings

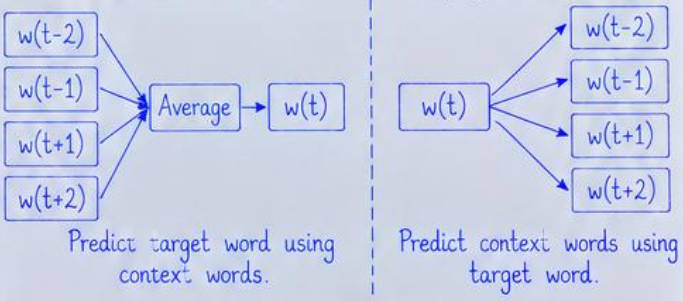


4.5 Popular Word Embedding Techniques

Technique	Year	Key Idea	Example Use Case
Word2Vec	2013	Predict surrounding words (CBOW, Skip-gram)	General NLP tasks
GloVe	2014	Use global word co-occurrence statistics	Semantic similarity
FastText	2016	Use subword information (character n-grams)	Handle rare words, morphology
BERT Embeddings	2018	Contextual embeddings from transformer	Context-aware representations

4.6 Word2Vec Overview

- Word2Vec learns word embeddings using a shallow neural network.
- Two main architectures:
  - CBOW (Continuous Bag of Words)
  - Skip-gram
- Learns embeddings by predicting context words (CBOW) or target word (Skip-gram).



4.7 GloVe Overview

- GloVe (Global Vectors) uses global word co-occurrence statistics from the entire corpus.
- It learns embeddings by factorizing the word co-occurrence matrix.
- Captures both local and global context information.
- Generally performs well on analogy and similarity tasks.

4.8 Example: Word Embeddings using Python (Gensim)

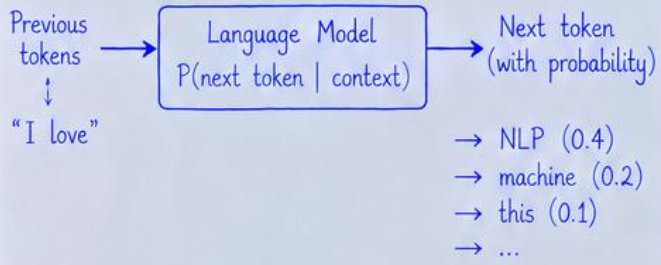
```
from gensim.models import Word2Vec
# Sample sentences
sentences = [
    ["I", "love", "NLP"],
    ["NLP", "is", "fun"],
    ["I", "enjoy", "machine", "learning"]
]
# Train Word2Vec model
model = Word2Vec(sentences, vector_size=100, window=2,
                 min_count=1, sg=1)  # sg=1 for Skip-gram
# Get word vector
vector = model.wv["NLP"]
print(vector)
# Find similar words
similar_words = model.wv.most_similar("NLP", topn=5)
print(similar_words)
```

Rahul Yadav

5. Language Models

5.1 What is a Language Model ?

- A language model (LM) assigns a probability to a sequence of tokens (words).
- It predicts the next token given the previous tokens in a sequence.
- It captures the patterns, grammar and semantic structure of a language.
- Used in many NLP tasks like text generation, autocomplete, translation, summarization, etc.



5.2 N-gram Language Models

- Predict the next word using the previous n-1 words (history).
- Based on the Markov assumption.
- Probability of next word:

$$P(w_t \mid w_{t-n+1}, \dots, w_{t-1})$$

- n = 1 → Unigram (no history)
- n = 2 → Bigram (1 previous word)
- n = 3 → Trigram (2 previous words)

5.3 Examples of N-grams

Type	Formula	Example
Unigram	$P(w_t)$	P("NLP")
Bigram	$P(w_t \mid w_{t-1})$	P("love" | "I")
Trigram	$P(w_t \mid w_{t-2}, w_{t-1})$	P("NLP" | "I love")

5.4 Markov Assumption

- Assumes the next word depends only on the previous n-1 words.
- Does not consider the full history of the sentence.
- Simplifies the model and reduces complexity.

$$P(w_t \mid w_1, w_2, \dots, w_{t-1}) \approx P(w_t \mid w_{t-n+1}, \dots, w_{t-1})$$

Limitations :

- Cannot capture long-range dependencies.
- Data sparsity problem (many n-grams not seen in training data).
- Ignores semantic meaning beyond local context.

5.5 Maximum Likelihood Estimation (MLE)

- Estimate probabilities from training data using token counts.
- For an n-gram model :

$$P(w_t \mid \text{context}) = \frac{\text{Count}(\text{context}, w_t)}{\text{Count}(\text{context})}$$

- context : previous n-1 words (e.g. w_{t-1} or w_{t-2}, w_{t-1})
- $\text{Count}(\text{context}, w_t)$: number of times the n-gram appears in training data.
- $\text{Count}(\text{context})$: number of times the context appears.

5.6 Smoothing Techniques

- Handle unseen n-grams (probability = 0) due to data sparsity.
- Common methods:

Method	Formula	Description
Add-one (Laplace)	$P(w_t \mid c) = \frac{\text{Count}(c, w_t) + 1}{\text{Count}(c) + V}$	Adds 1 to all counts (V = vocab size)
Add-k	$\frac{\text{Count}(c, w_t) + k}{\text{Count}(c) + kV}$	General form of add-one smoothing
Good-Turing	–	Adjusts counts based on unseen events
Kneser-Ney	–	Better performance for higher n-grams

5.7 Perplexity

- Measures how well a language model predicts a test set.
- Lower perplexity means better performance (on same data).

$$PP(W) = P(w_1, w_2, \dots, w_N)^{-1/N}$$

- W : sequence of N tokens.
- It is the inverse of the geometric mean probability.
- Often computed using log probabilities to avoid underflow.

5.8 Example: Bigram Language Model

Bigram (w_{t-1}, w_t)	Count	Probability
(I, love)	50	50 / 100 = 0.50
(I, like)	30	30 / 100 = 0.30
(I, hate)	20	20 / 100 = 0.20

$$P(\text{"love"} \mid \text{"I"}) = 50 / 100 = 0.50$$

5.9 N-gram vs Neural Language Models

Aspect	N-gram Models	Neural Language Models
Representation	Discrete counts	Dense embeddings
Context length	Fixed (n-1 words)	Can capture long context
Data sparsity	High	Much better
Ability to model semantics	Limited	Strong (context & meaning)
Examples	Unigram, Bigram, Trigram	RNN, LSTM, Transformer

5.10 Key Takeaways

- Language models assign probability to token sequences.
- N-gram models are simple but suffer from data sparsity.
- Smoothing techniques help handle unseen n-grams.
- Perplexity is used to evaluate language models.
- Neural language models (RNN, Transformer) overcome many limitations and are widely used in modern NLP.

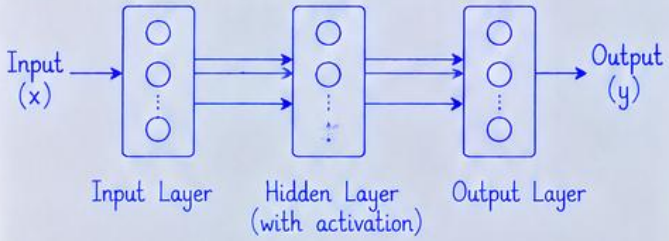
Rahul Yadav

6. Neural Networks for NLP

6.1 Role of Neural Networks in NLP

- Neural networks learn patterns from large text data automatically.
- They convert text into numerical representations and learn complex relationships.
- Different architectures are used for different NLP tasks (RNN, LSTM, GRU, Transformer, etc.).
- They help in capturing semantic meaning, context and long-range dependencies.

6.2 Basic Neural Network Architecture



- Input: numerical representation of text (e.g. embeddings)
- Hidden layers learn patterns using weights and biases
- Activation functions introduce non-linearity
- Output layer produces final prediction (e.g. next word, class label, etc.)

6.3 Activation Functions

Function	Formula	Range	Use Case
Sigmoid	$\sigma(x) = \frac{1}{1+e^{-x}}$	$(0, 1)$	Binary classification
Tanh	$\tanh(x) = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}}$	$(-1, 1)$	Hidden layers (RNN)
ReLU	$f(x) = \max(0, x)$	$[0, \infty)$	Most common
Leaky ReLU	$f(x) = \max(\alpha x, x)$	$(-\infty, \infty)$	Avoids dead neurons

6.4 Loss Functions (NLP)

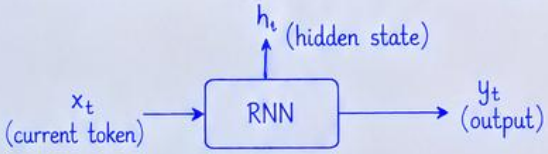
- Used to measure the difference between predicted output and actual output.
- Common loss functions :

Task	Loss Function	Formula
Language modeling	Cross-Entropy Loss	$L = -\sum_{i=1}^{V} y_i \log(\hat{y}_i)$
Text classification	Cross-Entropy Loss	$L = -\sum_{i=1}^{C} y_i \log(\hat{y}_i)$
Sequence generation	Cross-Entropy Loss (with teacher forcing)	$L = -\sum_{i=1}^{V} y_i \log(\hat{y}_i)$

- V : vocabulary size, C : number of classes
- y_i : true label, $\hat{y}_i$: predicted probability

6.5 Recurrent Neural Network (RNN)

- RNN processes text sequentially, one token at a time.
- It maintains a hidden state to remember previous information.
- Suitable for sequence data like text, speech, etc.



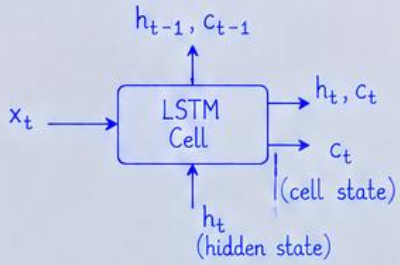
Hidden state update :

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$$

- h_t : hidden state at time t
- x_t : input at time t (token embedding)
- W_x, W_h : weight matrices
- b : bias vector

6.6 Long Short-Term Memory (LSTM)

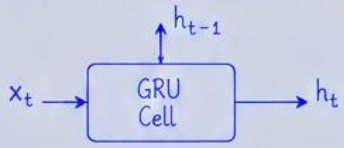
- Solves the vanishing gradient problem of RNN.
- Uses gates to control information flow.
- Can capture long-term dependencies in text.



Gates in LSTM :
- Forget gate (f_t) (what to forget)
- Input gate (i_t) (what to add)
- Output gate (o_t) (what to output)

6.7 Gated Recurrent Unit (GRU)

- Simpler than LSTM (fewer gates).
- Works similarly to LSTM but with update and reset gates.
- Performs well in many NLP tasks.



Gates in GRU :
- Update gate (z_t) (how much to keep)
- Reset gate (r_t) (how much to forget)

6.8 Comparison: RNN vs LSTM vs GRU

Aspect	RNN	LSTM	GRU
Vanishing gradient	Yes	No	Less severe
Gates	No	3 (f, i, o)	2 (z, r)
Complexity	Low	High	Medium
Training time	Faster	Slower	Faster
Performance	Basic tasks	Better (long-term)	Comparable to LSTM
Use case	Simple sequences	Complex NLP tasks	When faster training is needed

Rahul Yadav

7. Attention Mechanism

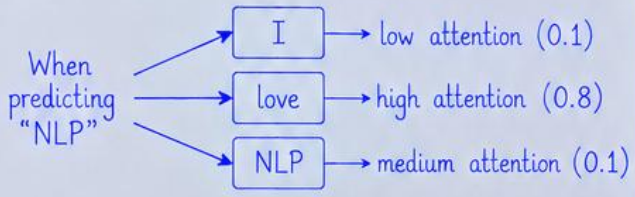
7.1 Why Attention is Needed ?

- In RNN/LSTM, it is difficult to capture long-range dependencies.
- All words are treated equally which is not ideal.
- Attention allows the model to focus on the most relevant words in the input sequence.
- It improves performance in tasks like translation, summarization, and question answering.

7.2 Intuition Behind Attention

- When predicting a word, not all previous words are equally important.
- Attention assigns a weight (score) to each word based on its relevance to the current word.
- More relevant words get higher attention weights.

Example : Translating " I love NLP"



7.3 Attention Score Calculation

- Attention score measures the similarity between two vectors (query and key).
- Common method: Scaled Dot-Product Attention.
- Steps :
  1. Compute similarity between query and keys
  2. Scale the scores
  3. Apply Softmax to get attention weights
  4. Multiply weights with values to get output

7.4 Scaled Dot-Product Attention

- The most commonly used attention mechanism in Transformers.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- Q : Query matrix (shape: $n \times d_k$)
- K : Key matrix (shape: $n \times d_k$)
- V : Value matrix (shape: $n \times d_v$)
- d_k : Dimension of keys/queries
- QK^T : Similarity scores between queries and keys
- $\sqrt{d_k}$: Scaling factor to prevent large values
- Softmax : Convert scores to probabilities
- Result : Weighted sum of value vectors

7.5 Step by Step Example

Consider a simple example with 3 tokens and $d_k = 2$.

$$Q = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix} \quad K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix} \quad V = \begin{bmatrix} 1 & 2 \\ 2 & 0 \\ 0 & 1 \end{bmatrix}$$

1. Compute QK^T

$$QK^T = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 2 & 1 & 2 \end{bmatrix}$$

2. Scale by $\sqrt{d_k}$ $(d_k = 2)$

$$\text{Scaled Scores} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 2 & 1 & 2 \end{bmatrix}$$

3. Apply Softmax (row-wise)

Attention Weights (α) = softmax (Scaled Scores)

$$\alpha = \begin{bmatrix} 0.40 & 0.20 & 0.40 \\ 0.20 & 0.40 & 0.40 \\ 0.25 & 0.25 & 0.50 \end{bmatrix} \leftarrow \text{Each row sums to 1}$$

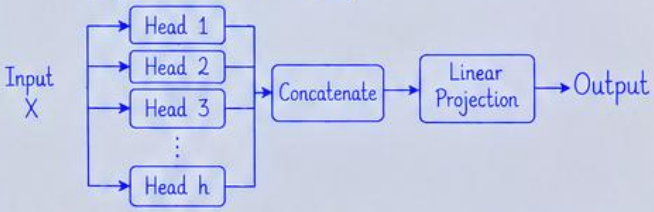
4. Compute Output

$$\text{Output} = \alpha V = \begin{bmatrix} 0.40 & 0.20 & 0.40 \\ 0.20 & 0.40 & 0.40 \\ 0.25 & 0.25 & 0.50 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 2 & 0 \\ 0 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} 0.80 & 1.00 \\ 1.20 & 0.60 \\ 0.75 & 1.00 \end{bmatrix}$$

7.6 Multi-Head Attention

- Instead of using a single attention head, multiple attention heads are used in parallel.
- Each head learns different relationships from the data.
- Outputs from all heads are concatenated and projected into final output.



7.7 Advantages of Attention Mechanism

- Captures long-range dependencies effectively.
- Allows parallelization (faster training than RNN).
- Focuses on relevant parts of the input.
- Works well for various NLP tasks.

7.8 Key Takeaways

- Attention computes relevance between words.
- Scaled Dot-Product Attention is the standard method.
- Multi-Head Attention improves model performance.
- Attention is the core component of the Transformer model.

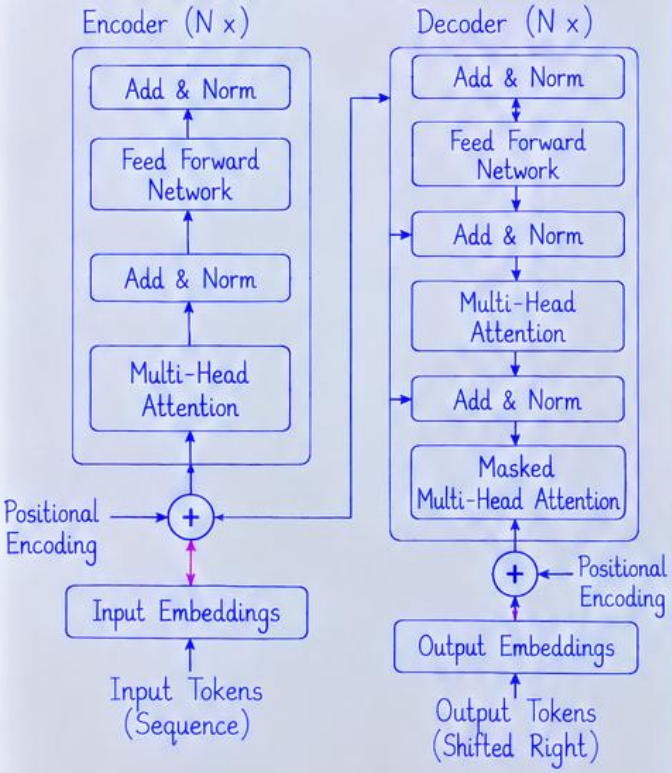
Rahul Yadav

8. Transformer Architecture

8.1 What is Transformer ?

- Transformer is a deep learning architecture based on self-attention mechanism.
- It was introduced in the paper "Attention is All You Need" (2017) by Vaswani et al.
- It removed recurrence and convolution and uses only attention.
- It is highly parallelizable and effective for handling long-range dependencies.

8.2 Overall Architecture



8.3 Input Embeddings

- Converts tokens (words) into dense vectors.
- Each token is mapped to a vector of size d_{model} (e.g. 512, 768, 1024).

Token	Token ID	Embedding Vector (d = 4)
I	101	[0.12, 0.03, -0.11, 0.88]
love	2003	[0.21, -0.34, 0.56, 0.12]
NLP	17953	[-0.42, 0.87, 0.21, -0.18]

8.4 Positional Encoding

- Since transformer does not have recurrence, it has no notion of token order.
- Positional encoding adds information about the position of each token.
- Uses sine and cosine functions of different frequencies.

8.5 Positional Encoding Formula

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

$$PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

- pos : position of token (0, 1, 2, ..,)
- i : dimension index (0, 1, 2, ...)
- d_{model} : embedding dimension (e.g. 512, 768)

8.6 Self-Attention Mechanism

- Computes attention scores between all tokens in the input sequence.
- Each token attends to every other token.
- Helps to capture long-range dependencies.

Steps :

1. Create Query (Q), Key (K) and Value (V) matrices.
2. Compute attention scores using scaled dot-product.
3. Apply softmax to get attention weights.
4. Multiply weights with value vectors to get output.

8.7 Scaled Dot-Product Attention

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- Q : Query matrix (shape: n x d_k)
- K : Key matrix (shape: n x d_k)
- V : Value matrix (shape: n x d_v)
- d_k : Dimension of keys/queries
- QK^T : Similarity scores between tokens
- $\sqrt{d_k}$: Scaling factor
- Result : Weighted sum of value vectors

8.8 Attention Computation (Example)

Suppose we have 3 tokens and $d_k = 2$.

$$Q = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix} \quad K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix} \quad V = \begin{bmatrix} 1 & 2 \\ 2 & 0 \\ 0 & 1 \end{bmatrix}$$

1. Compute QK^T

$$QK^T = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 2 \end{bmatrix}$$

2. Scale by $\sqrt{d_k}$ ($\sqrt{2}$) and apply softmax (row-wise)

$$\text{Attention Weights} = \begin{bmatrix} 0.40 & 0.20 & 0.40 \\ 0.20 & 0.40 & 0.40 \\ 0.25 & 0.25 & 0.50 \end{bmatrix}$$

3. Output = Attention Weights × V

$$\text{Output} = \begin{bmatrix} 0.40 & 0.20 & 0.40 \\ 0.20 & 0.40 & 0.40 \\ 0.25 & 0.25 & 0.50 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 2 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 0.80 & 1.00 \\ 1.20 & 0.60 \\ 0.75 & 1.00 \end{bmatrix}$$

Rahul Yadav

9. Pre-Trained Language Models

9.1 What is a Pre-trained Language Model ?

- A model trained on large text corpora before being adapted to downstream tasks.
- Learns grammar, semantics, context and patterns from data.
- Examples : BERT, GPT, T5.

9.2 Main Pre-training Approaches

- **MLM (Masked Language Modeling) :** predict masked tokens from both-side context. Example : BERT
- **CLM (Causal Language Modeling) :** predict the next token using left context. Example : GPT
- **Seq2Seq denoising :** reconstruct corrupted text. Example : T5

(1) Masked Language Modeling (BERT)



(2) Causal Language Modeling (GPT)



(3) Seq2Seq Denoising (T5)



9.3 BERT vs GPT vs T5

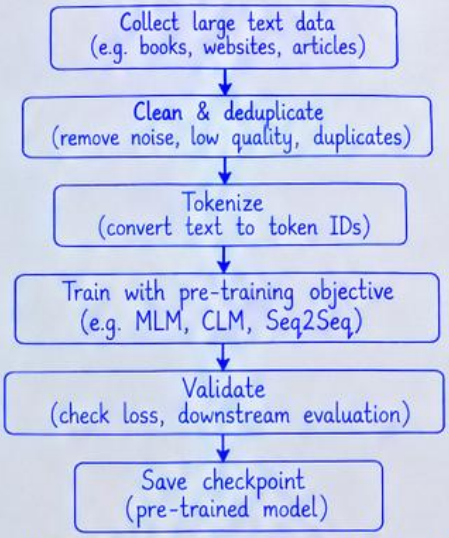
Model	Architecture	Typical Use
BERT	Encoder-only	Text classification, NER, sentiment analysis
GPT	Decoder-only	Text generation, conversation, summarization
T5	Encoder-decoder	Translation, summarization, question answering

9.4 Tokenization and Input

- Text is first converted into tokens using a tokenizer (e.g. WordPiece, BPE, SentencePiece).
- Each token is mapped to a unique token ID and then converted into dense embeddings (vectors).
- Special tokens depend on model family (e.g. [CLS], [SEP] in BERT, <s>, </s> in T5).



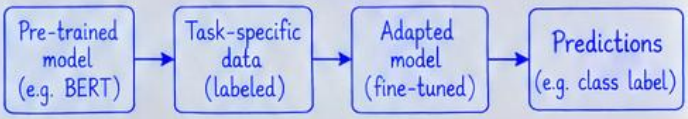
9.5 How Pre-training Works



- Split data carefully into train/validation/test sets to avoid data leakage.
- Training minimizes a loss such as cross-entropy over the predicted tokens.

9.6 Downstream Fine-tuning

- Add a task specific head (e.g. classification layer) or use prompt-based approach.
- Train on labeled / task-specific data and evaluate on held-out validation/test data.



9.7 Practical Example : Text Classification



9.8 Common Interview Questions

- Why pre-train before fine-tuning ?
  Reuse broad language knowledge and reduce task-specific data needs.
- Can a pre-trained model be biased ?
  Yes, training data and objectives can encode bias.
- Does pre-training guarantee factual accuracy ?
  No, outputs still need evaluation and grounding (e.g. RAG, tool use, human review).

9.9 Key Takeaways

- Pre-training learns reusable representations from large datasets.
- MLM uses context on both sides; CLM predicts subsequent tokens.
- Choose model architecture based on task, latency, cost and quality requirements.

Rahul Yadav

10. Fine-Tuning and Transfer Learning

10.1 What is Fine-tuning ?

- Fine-tuning means taking a pre-trained model and further training it on a specific dataset for a downstream task.
- It adjusts the model's weights to better perform on the new task.
- We usually use a small learning rate and fewer epochs.
- Examples : BERT for sentiment analysis, T5 for text summarization.

10.2 What is Transfer Learning ?

- Transfer learning is a technique where knowledge learned from one task is transferred to another task.
- Pre-trained models already capture general language understanding from large data, which can be reused.
- It helps to achieve good performance even with small task-specific data.

10.3 Fine-tuning vs Transfer Learning

Aspect	Fine-tuning	Transfer Learning
Meaning	Further training a pre-trained model on specific dataset	Using knowledge from a pre-trained model for a new task
Data requirement	Small to medium labeled data	Works well even with very small data
Model weights	Updates model weights	Reuses pre-trained weights
Objective	Task-specific optimization	Leverage general knowledge
Example	Fine-tuning BERT for sentiment analysis	Using GPT for a custom chatbot

10.4 Fine-tuning Process (Workflow)



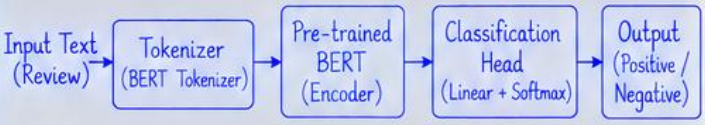
10.5 Types of Fine-tuning

- **Full Fine-tuning** : Update all model parameters.
- **Layer-wise Fine-tuning** : Update only top layers.
- **Adapter-based Fine-tuning** : Add small adapter layers and keep base model frozen.
- **LoRA (Low-Rank Adaptation)** : Train only low-rank matrices, very parameter efficient.
- **Prompt Tuning** : Learn soft prompts instead of full model weights.
- **Prefix Tuning** : Learn prefix embeddings for generation tasks.

10.6 Popular Fine-tuning Techniques

Technique	Idea	Parameters Trained	Use Case
Full Fine-tuning	Update all weights	All	Large dataset, high-quality data
Layer-wise Fine-tuning	Update top layers	Few layers	Smaller dataset
Adapters	Add small trainable adapter layers	Only adapter layers	Parameter efficient
LoRA	Low-rank matrices instead of full weights	Very few parameters	Large models (e.g. Llama, GPT)
Prompt Tuning	Learn soft prompts	Prompt embeddings	Few-shot tasks
Prefix Tuning	Add prefix embeddings	Prefix parameters	Text generation

10.7 Example : Fine-tuning BERT for Sentiment Analysis



Steps :

1. Prepare labeled dataset (text, label).
2. Tokenize input text (input_ids, attention_mask).
3. Add classification head (linear layer).
4. Fine-tune using cross-entropy loss.
5. Evaluate using accuracy, precision, recall, F1 score.

10.8 Key Takeaways

- Fine-tuning adapts pre-trained models to specific downstream tasks.
- Transfer learning reduces training time and data requirements.
- Techniques like LoRA, Adapters, Prompt Tuning are parameter efficient.
- Choose the right fine-tuning method based on dataset size, compute resources and task requirements.
- Proper evaluation is important to measure real performance.

Rahul Yadav

11. NLP Tasks and Applications

11.1 Introduction

- NLP has a wide range of tasks and real-world applications.
- These tasks focus on understanding, generating and manipulating human language.
- With the help of deep learning and pre-trained models, performance in these tasks has improved significantly.
- Examples : text classification, NER, summarization, translation, question answering, etc.

11.2 Text Classification

- Assign a predefined category or label to a text.
- Used for sentiment analysis, spam detection, topic classification, etc.
- It is usually a supervised learning task.

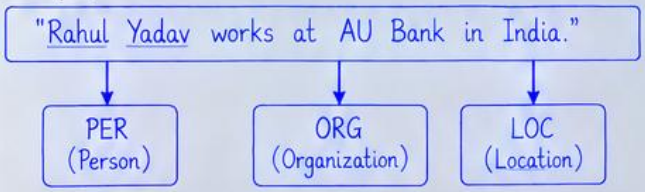
Example : Sentiment Analysis



11.3 Named Entity Recognition (NER)

- Identifies and classifies important entities in text such as person, organization, location, date, etc.
- Common entity types : PER, ORG, LOC, DATE, MISC.
- Used in search, information extraction, chatbots, etc.

Example :



11.4 Text Summarization

- Generate a shorter version of a longer text while keeping the key information.
- Two types :
  - Extractive Summarization : select important sentences from the original text.
  - Abstractive Summarization : generate a new summary in your own words.

Example :

Input Text : "AU Small Finance Bank reported a 25% increase in net profit in Q1 FY26. The growth was driven by strong loan disbursements and improved asset quality. The bank expects this momentum to continue in the coming quarters."

↓

Summary : "AU Bank reported 25% higher net profit in Q1 FY26, driven by strong loan growth and better asset quality."

11.5 Machine Translation

- Convert text from one language to another.
- Uses encoder-decoder architecture (Transformer).
- Examples : Google Translate, multilingual chatbots.

Example :



11.6 Question Answering (QA)

- Given a context and a question, the model predicts the correct answer.
- Types :
  - Extractive QA : selects answer span from context.
  - Generative QA : generates answer in natural language.
- Used in chatbots, search engines, document Q&A, etc.

Example :

Context :	"AU Small Finance Bank was founded in 1996 and is headquartered in Jaipur, Rajasthan."
Question :	"Where is AU Small Finance Bank headquartered?"
Answer :	"Jaipur, Rajasthan"

11.7 Text Generation

- Generate human-like text based on a given prompt.
- Examples : story generation, chatbots, product descriptions, code generation, etc.
- Uses large language models like GPT, T5, Llama, etc.

Example :

Prompt : "Write a short message about digital banking."

↓

Generated Output : "Digital banking makes your financial transactions faster, safer and more convenient anytime, anywhere."

11.8 Other Important NLP Tasks

Task	Description
Text Similarity	Measure similarity between two texts (semantic search, duplicate detection)
Paraphrase Detection	Identify if two sentences have the same meaning
Text Clustering	Group similar documents without labels
Topic Modeling	Discover hidden topics in a collection of texts
Text Generation (Controlled)	Generate text with specific constraints (e.g. tone, style, length)
Conversational AI	Build intelligent chatbots and virtual assistants

11.9 Key Takeaways

- NLP has many real-world tasks and applications.
- Pre-trained models significantly improve performance.
- Each task has different techniques and evaluation metrics.
- NLP is widely used in products like search, chatbots, translation, summarization and recommendation systems.

Rahul Yadav

12. Text Generation

12.1 What is Text Generation ?

- Text generation is the task of automatically generating human-like text based on a given input or context.
- It uses deep learning models to predict the next word (or sequence of words).
- Applications include chatbot responses, content creation, code generation, summarization, story writing, etc.

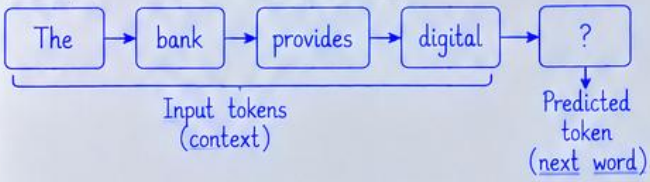
12.2 Types of Text Generation

- Autoregressive Generation : generate text sequentially from left to right (e.g. GPT).
- Sequence-to-Sequence Generation : encoder-decoder approach (e.g. T5, BART).
- Controlled Generation : generate text with specific constraints (e.g. length, style, topic).
- Conditional Generation : generate text based on additional input (e.g. prompt, image, data).

12.3 Autoregressive Text Generation (GPT)

- Predicts the next token based on previous tokens.
- Uses only decoder architecture (Transformer).
- Trained with next token prediction objective.
- Generates text one token at a time.

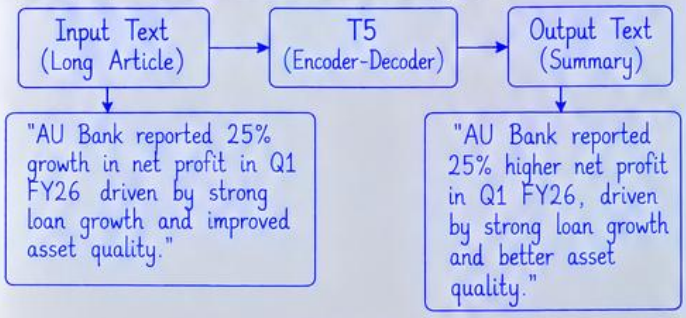
Example : Next Token Prediction



12.4 Sequence-to-Sequence Text Generation (T5)

- Uses encoder-decoder architecture.
- Converts input text to a fixed representation (encoder) and then generates output text (decoder).
- Used for tasks like translation, summarization, paraphrasing, etc.

Example : Text Summarization using T5



12.5 Conditional Text Generation

- Generate text based on additional conditions like topic, style, sentiment, length, etc.
- Used in creative writing, marketing content, chatbots, etc.
- **Example** : Generate a positive review for a banking app.

12.6 Decoding Strategies

Strategy	Description	Pros	Cons
Greedy Search	Always pick the most probable next token	Fast and simple	Less diverse output
Beam Search	Keep top-k sequences at each step	Better quality output	Computationally expensive
Top-k Sampling	Sample from top k probable tokens	More diverse text	May generate less relevant text
Top-p (Nucleus) Sampling	Sample from smallest set of tokens with cumulative probability p	Good balance of quality and diversity	Needs tuning of p value
Temperature Sampling	Control randomness using temperature	More creative output	High temperature may generate nonsensical text

12.7 Example : Text Generation using GPT

- Input Prompt : "Write a short message for customers about digital banking."
- Generated Output :

> "Experience fast, secure and convenient banking with AU Digital Banking. Manage your accounts, transfer funds, pay bills and more — anytime, anywhere."

12.8 Applications of Text Generation

- Chatbots and virtual assistants (customer support)
- Content creation (blogs, social media posts)
- Email and message generation
- Code generation (e.g. GitHub Copilot)
- Summarization and paraphrasing
- Personalized marketing messages
- Educational content generation
- Creative writing (stories, poems, etc.)

12.9 Key Takeaways

- Text generation predicts the next token based on context.
- GPT uses autoregressive decoding, while T5 uses encoder-decoder architecture.
- Different decoding strategies control the quality and diversity of generated text.
- Text generation has wide real-world applications in banking, education, marketing, customer support, and more.
- Proper prompt design and decoding parameters significantly impact the output quality.

Rahul Yadav

13. Information Retrieval and Search

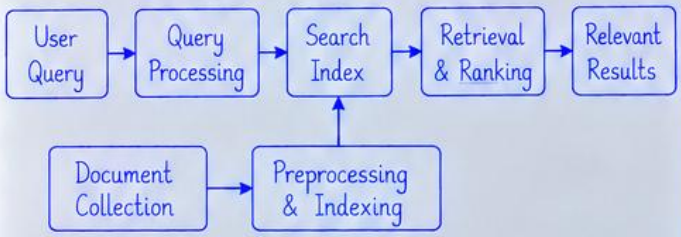
13.1 What is Information Retrieval ?

- Information Retrieval (IR) is the process of finding relevant information from a large collection of documents based on a user query.
- It focuses on searching and ranking the most relevant results.
- Used in search engines, chatbots, recommender systems, enterprise search, etc.

13.2 Key Components of an IR System

- Document Collection : Set of documents (text, PDFs, web pages, etc.).
- Query : User's search input
- Indexing : Organize and store documents for fast search
- Retrieval Model : Find relevant documents
- Ranking : Order results by relevance
- Evaluation : Measure the quality of results

13.3 IR System Architecture



13.4 Document Indexing

- Converts documents into a format that can be searched efficiently.
- The most common structure is an Inverted Index.
- Inverted Index maps each term to the list of documents containing that term.

Example : Inverted Index

Term	Posting List (Document IDs)
bank	[1, 3, 5]
loan	[2, 3]
finance	[1, 2, 4]
digital	[3, 4]

13.5 Text Preprocessing for IR

- Tokenization : Split text into tokens.
- Lowercasing : Convert to lowercase.
- Stopword Removal : Remove common words (e.g. the, is, and).
- Stemming : Reduce words to root form (e.g. banking → bank).
- Lemmatization : Convert to base form (e.g. better → good).
- Handle special characters, numbers, etc.

13.6 Retrieval Models

- Boolean Retrieval : Exact matching using AND, OR, NOT (e.g. bank AND loan).
- Vector Space Model : Represent documents and query as vectors (TF-IDF, cosine similarity).
- Probabilistic Model : Estimate probability of relevance (e.g. BM25).
- Neural Retrieval : Use embeddings and deep learning models (e.g. BERT, Sentence-BERT).

13.7 TF-IDF (Term Frequency - Inverse Document Frequency)

- TF (Term Frequency) : how often a term appears in a document.
- IDF (Inverse Document Frequency) : how rare the term is across all documents.
- TF-IDF gives higher weight to important and rare terms.

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \text{IDF}(t) \qquad \text{IDF}(t) = \log\left(\frac{N}{df(t)}\right)$$

N : total number of documents
df(t) : number of documents containing term t

13.8 BM25 (Best Performing IR Model)

- An improved version of TF-IDF.
- Considers term frequency saturation and document length normalization.
- Widely used in real-world search engines (e.g. Elasticsearch).

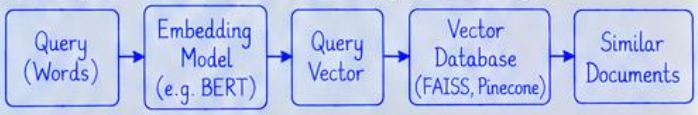
$$\text{Score}(d, q) = \sum \text{IDF}(t) \times \frac{\text{TF}(t, d) \times (k_1 + 1)}{\text{TF}(t, d) + k_1 \times \left(1 - b + b \times \left[\frac{|d|}{\text{avgdl}}\right]\right)}$$

k_1 : term frequency saturation parameter (usually 1.2 – 2.0)
b : document length normalization parameter (usually 0.75)

13.9 Dense Retrieval using Embeddings

- Convert text into dense vectors using models like BERT, Sentence-BERT, OpenAI embeddings.
- Compute similarity using cosine similarity.
- Useful for semantic search (understands meaning, not just exact words).

Example : Semantic Search using Embeddings



13.10 Practical Applications

- Web search engines (Google, Bing)
- Enterprise document search
- Chatbots with document search (RAG)
- E-commerce product search
- Legal and medical document retrieval
- Recommender systems
- Knowledge management systems

13.11 Key Takeaways

- IR helps find relevant information from large data.
- Indexing and ranking are core components.
- TF-IDF and BM25 are traditional methods.
- Embeddings enable semantic search.
- IR is widely used in search engines, chatbots, and enterprise apps.

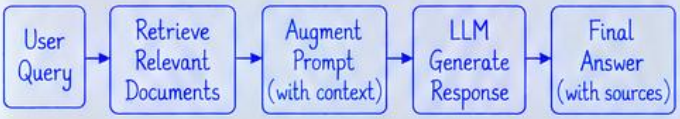
Rahul Yadav

14. RAG (Retrieval-Augmented Generation)

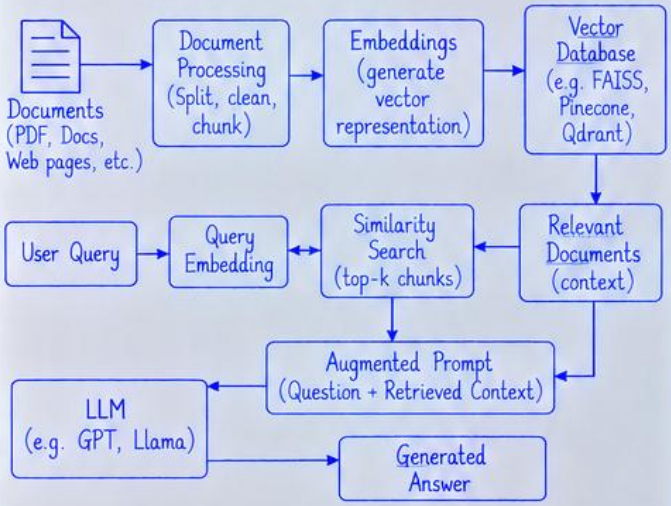
14.1 What is RAG ?

- RAG (Retrieval-Augmented Generation) is a technique that combines information retrieval with text generation.
- It retrieves relevant information from external knowledge sources (e.g. documents, databases) and uses it as context for a language model.
- It helps in generating more accurate, up-to-date and domain-specific responses.
- RAG reduces hallucination and provides factually grounded answers.

14.2 How RAG Works ? (High Level Flow)



14.3 RAG Architecture



14.4 Steps in RAG Pipeline

1 Data Ingestion	• Load documents (PDF, DOCX, web, etc.) • Extract text from files
2 Text Splitting	• Split text into smaller chunks • Use fixed size or semantic split
3 Embedding Generation	• Convert text chunks into vectors using embedding models • Example: all-MiniLM-L6-v2, OpenAI embeddings
4 Store in Vector DB	• Store embeddings with metadata • Examples: FAISS, Pinecone, Qdrant, Weaviate
5 Retrieve Relevant Chunks	• Convert user query to embedding • Find top-k similar chunks (similarity search)
6 Augment Prompt	• Combine user query + retrieved context into a prompt • Use a proper prompt template
7 Generate Response	• Send prompt to LLM (e.g. GPT, Llama) • Get final answer with sources

14.5 Chunking Strategies

Strategy	Description	Use Case	Pros	Cons
Fixed Size Chunking	Split text by fixed number of characters/tokens	Simple documents	Easy to implement	May break semantic meaning
Recursive Chunking	Split using hierarchy (para, sentence, token)	General purpose	Better context preservation	Slightly complex
Semantic Chunking	Split based on meaning using embeddings	Technical/ long documents	High quality chunks	Computationally expensive

14.6 Embedding Models (Popular Options)

Model	Dimension	Use Case	Notes
all-MiniLM-L6-v2	384	General purpose	Lightweight, fast, open source
OpenAI text-embedding-3-small	1536	High quality embeddings	Good for production
bge-base-en	768	Better for retrieval tasks	Good performance
Instructor-xl	768	Instruction based embeddings	Handles queries well

14.7 Vector Databases

Vector DB	Type	Key Features	Use Case
FAISS	Open Source	Fast similarity search, local setup	Small to medium scale
Pinecone	Managed (Cloud)	Scalable, high availability	Production applications
Qdrant	Open Source	Filtering, metadata support	Flexible and powerful
Weaviate	Open Source	Hybrid search, GraphQL	Enterprise use cases
Milvus	Open Source	Scalable, high performance	Large scale applications

14.8 Prompt Template for RAG

```
You are a helpful assistant. Use the following context
to answer the question. If the answer is not in the context,
say you don't know.
Context:
{context}
Question: {question}
Answer:
```

14.9 Advantages and Limitations

Advantages	Limitations
• More accurate and factual answers • Uses latest and domain-specific data • Reduces hallucination • Can work with private/enterprise data • Provides source references • Better for knowledge-intensive tasks	• Requires document processing and indexing • Increased system complexity • Latency is higher than direct LLM • Quality depends on retrieval results • Needs proper chunking and embedding strategy

14.10 Real-world Applications

- Enterprise knowledge search (internal documents)
- Customer support chatbots
- Research and document Q&A
- Legal and medical domain assistants
- Product documentation and technical support

14.11 Key Takeaways

- RAG combines retrieval and generation for better results.
- Good chunking, embeddings and vector DB are important.
- Use proper prompt template and top-k retrieval.
- RAG is widely used in real-world AI applications.

Rahul Yadav

15. Evaluation Metrics for NLP

15.1 Introduction

- Evaluation metrics help us measure the performance of NLP models.
- Different tasks use different metrics.
- Metrics tell us how close the model output is to the expected (ground truth) output.
- Both automatic metrics and human evaluation are important.

15.2 Types of Evaluation Metrics

- Classification Metrics (Accuracy, Precision, Recall, F1)
- Generation Metrics (BLEU, ROUGE, BERTScore, etc.)
- Ranking Metrics (MRR, nDCG, MAP)
- Semantic Similarity Metrics (Cosine Similarity, etc.)
- Human Evaluation (Fluency, Coherence, Relevance)

15.3 Classification Metrics

- Used for tasks like sentiment analysis, text classification, NER, etc.
- Based on True Positive (TP), True Negative (TN), False Positive (FP), False Negative (FN).

Confusion Matrix

Actual \ Predicted	Positive	Negative
Positive	TP	FN
Negative	FP	TN

Accuracy

$$\text{Accuracy} = \frac{(TP + TN)}{(TP + TN + FP + FN)}$$

Precision

$$\text{Precision} = \frac{TP}{TP + FP}$$

Recall

$$\text{Recall} = \frac{TP}{TP + FN}$$

F1 Score

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

15.4 Generation Metrics Overview

- Used for tasks like text summarization, machine translation, text generation, etc.
- Compare the generated text with a reference text.
- Use n-gram overlap, semantic similarity, or embeddings.

15.5 BLEU (Bilingual Evaluation Understudy)

- Widely used for machine translation.
- Measures n-gram overlap between generated and reference text.
- Considers precision (not recall).
- Range: 0 to 1 (higher is better).

$$\text{BLEU} = \text{BP} \times \exp\left(\frac{1}{n}\sum_{i=1}^{n} w_i \log p_i\right)$$

BP : Brevity Penalty
p_i : Modified n-gram precision
w_i : Weights (usually equal)

15.6 ROUGE (Recall-Oriented Understudy for Gisting Evaluation)

- Commonly used for text summarization.
- Measures n-gram overlap, longest common subsequence, and word pairs.
- Focuses on recall (how much of the reference is covered).
- Popular variants: ROUGE-1, ROUGE-2, ROUGE-L.

Metric	Description	Use Case
ROUGE-1	Unigram overlap	Measures word-level overlap
ROUGE-2	Bigram overlap	Measures phrase-level overlap
ROUGE-L	Longest Common Subsequence (LCS)	Considers sentence structure

15.7 METEOR (Metric for Evaluation of Translation)

- Improved version of BLEU.
- Considers exact match, stem match, synonym match and paraphrases.
- Uses both precision and recall.
- Range: 0 to 1 (higher is better).

$$\text{METEOR} = 10 \times \frac{(\text{Precision} \times \text{Recall})}{\alpha \times \text{Precision} + (1-\alpha) \times \text{Recall}}$$

α : weight parameter (usually 0.9)

15.8 BERTScore

- Uses contextual embeddings from pre-trained models (e.g. BERT) to compute similarity.
- Compares semantic similarity instead of exact word match.
- More robust than BLEU and ROUGE.
- Range: 0 to 1 (higher is better).

$$\text{Precision} = \frac{1}{|X|}\sum_{x \in X} \max_{y \in Y} \text{sim}(x, y)$$

$$\text{Recall} = \frac{1}{|Y|}\sum_{y \in Y} \max_{x \in X} \text{sim}(y, x)$$

$$F1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

15.9 Semantic Similarity Metrics

- Used to measure semantic similarity between two texts.
- Common methods: Cosine Similarity, Sentence-BERT, SBERT, OpenAI embeddings.
- Range: −1 to 1 (or 0 to 1 for embeddings, higher is better).

$$\text{Cosine Similarity} = \frac{A \cdot B}{\|A\| \times \|B\|}$$

A, B : vector embeddings
$A \cdot B$: dot product
$\|A\|$: magnitude of vector A

15.10 Human Evaluation

- Automatic metrics may not capture fluency, coherence and factual correctness.
- Human evaluation is important for generative tasks.
- Common evaluation criteria :

Criteria	Description	Scale (1–5)
Fluency	Grammar and naturalness	1 (poor) – 5 (excellent)
Coherence	Logical flow of the text	1 – 5
Relevance	Relevance to the input/context	1 – 5
Factuality	Correct and factual information	1 – 5

15.11 Key Takeaways

- Choose the right metric based on the NLP task.
- Use multiple metrics for a better evaluation.
- Automatic metrics + human evaluation gives the best results.
- Understand the strengths and limitations of each metric.

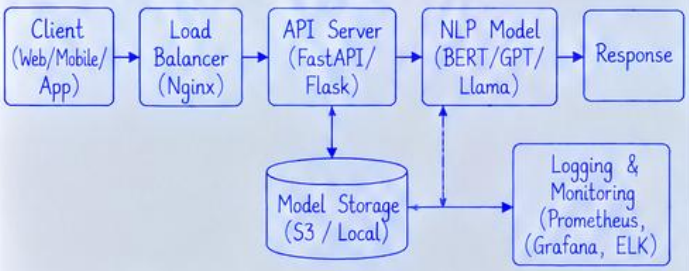
Rahul Yadav

16. Deployment and MLOps for NLP

16.1 Introduction

- Deployment makes NLP models available for real-world use.
- MLOps helps in automating, monitoring and managing the complete model lifecycle.
- It ensures reliable, scalable, and efficient deployment of NLP applications.
- Covers model serving, scaling, monitoring, logging, versioning and retraining.

16.2 Deployment Architecture



16.3 Model Serving Options

Option	Description	Use Case	Pros	Cons
FastAPI	Python web framework	Lightweight model serving	Fast, easy to use	Limited built-in MLOps
Flask	Simple web framework	Small/medium projects	Easy, flexible	Not ideal for large scale
TensorFlow Serving	Official TensorFlow model server	Production ML models	High performance	Complex setup
TorchServe	Model server for PyTorch	PyTorch models	Easy scaling	Limited features
Hugging Face Text Generation Inference (TGI)	Optimized server for LLMs	Large language models (GPT, Llama, etc.)	High throughput, GPU support	Resource intensive

16.4 Containerization with Docker

- Docker helps to package the application and model with all dependencies.
- Ensures consistent environment across development, testing and production.
- Easy to deploy on cloud or on-premise servers.
- Example Dockerfile for a FastAPI NLP model :

```
FROM python:3.10-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
```

16.5 Model Deployment on Cloud

Platform	Service	Use Case	Notes
AWS	SageMaker	Train, deploy, manage NLP models	Supports auto-scaling, monitoring
Google Cloud	Vertex AI	End-to-end ML platform	Easy deployment and model monitoring
Azure	ML Service	Deploy and manage models	Integrates with MLOps tools
Hugging Face	Inference Endpoints	Deploy LLMs easily	Managed, scalable infrastructure

16.6 Scaling and Optimization

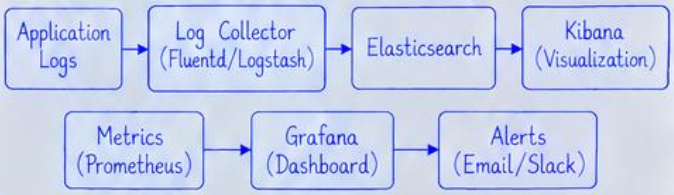
- Use horizontal scaling with multiple instances.
- Use load balancer to distribute traffic.
- Optimize model size (quantization, pruning).
- Use GPU/TPU for large models.
- Implement caching for repeated queries.
- Use asynchronous processing for high throughput.

16.7 Model Optimization Techniques

Technique	Description	Benefit
Quantization	Reduce precision (FP32 → INT8)	Smaller size, faster inference
Pruning	Remove less important weights	Reduced model size
Knowledge Distillation	Train smaller model from larger model	Faster and lightweight
ONNX Conversion	Convert model to ONNX format	Better performance across platforms
Batching	Process multiple requests together	Higher throughput

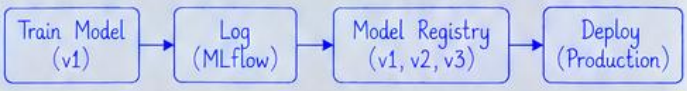
16.8 Monitoring and Logging

- Monitor API latency, throughput, error rate.
- Track model performance (accuracy, drift).
- Log input requests and model responses.
- Use tools like Prometheus, Grafana, ELK Stack.
- Set up alerts for failures and degraded performance.
- Monitor GPU/CPU usage and memory.



16.9 Model Versioning

- Keep track of different model versions.
- Use tools like MLflow, DVC or Hugging Face Model Hub.
- Store models in a versioned manner.
- Helps in rollback if a new model performs poorly.



16.10 Continuous Integration and Deployment (CI/CD)

- Automate testing, building and deployment of NLP models.
- Use tools like GitHub Actions, GitLab CI, Jenkins.
- Run unit tests and model evaluation before deployment.
- Deploy automatically to staging and production.



16.11 Key Takeaways

- Deploy NLP models using FastAPI, Flask or specialized servers.
- Use Docker and cloud platforms for scalability.
- Optimize models for faster and cost-efficient inference.
- Implement monitoring, logging and alerting.
- Use versioning and CI/CD for reliable deployment.
- MLOps ensures smooth and production-ready NLP applications.

Rahul Yadav

17. Advanced Topics in NLP

17.1 Introduction

- Advanced NLP techniques help build more powerful, flexible and multi-modal AI systems.
- These topics go beyond basic text processing and cover latest research and real-world innovations.
- Important for building next-generation applications like intelligent assistants, document understanding, and multimodal AI.

17.2 Prompt Engineering

- Prompt engineering is the art of designing effective prompts to get better responses from LLMs.
- A good prompt provides clear instructions, context, examples and expected output format.
- Techniques: zero-shot, few-shot, chain-of-thought, role prompting, instruct prompting, etc.

Example: Prompt Engineering for Summarization

Prompt:
You are a helpful assistant. Summarize the following article in 3 bullet points for a non-technical audience.
Text: {article}
Output:
- Point 1
- Point 2
- Point 3

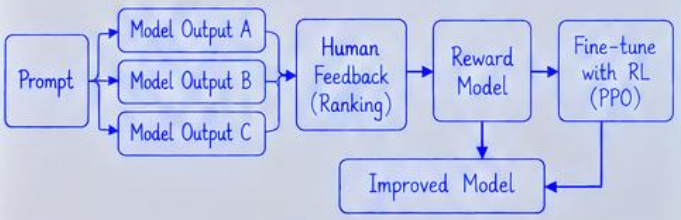
17.3 Instruction Tuning

- Instruction tuning trains LLMs to follow natural language instructions.
- Uses datasets like Alpaca, FLAN, ShareGPT, etc.
- Helps model understand and follow human intent better.
- Result: more aligned, helpful and safe responses.



17.4 Reinforcement Learning from Human Feedback (RLHF)

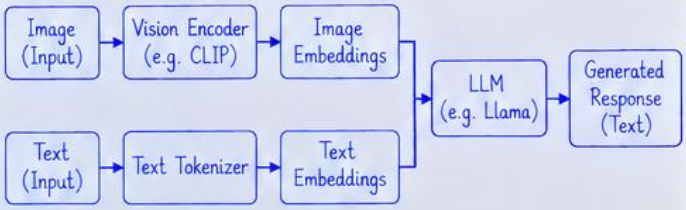
- RLHF uses human feedback to improve model responses.
- Humans rank model outputs and a reward model is trained.
- Model is further trained using Reinforcement Learning (e.g. PPO) to generate better responses.



17.5 Multimodal NLP (Text + Image/Audio)

- Multimodal models can understand and generate content from multiple modalities (text, image, audio).
- Examples: LLaVA, GPT-4V, Whisper, Qwen-VL.
- Use cases: image captioning, visual Q&A, document understanding, audio transcription, video analysis.

17.6 Multimodal Architecture (e.g. LLaVA)



17.7 Tool Use and Function Calling

- LLMs can use external tools and APIs to perform actions like search, get real-time data, run code, etc.
- Function calling allows the model to call predefined functions with arguments.
- Examples: OpenAI Function Calling, LangChain Tools, AutoGPT, ReAct.

Example: Function Calling

User: "What is the weather in Delhi?"
Model Output (Function Call):

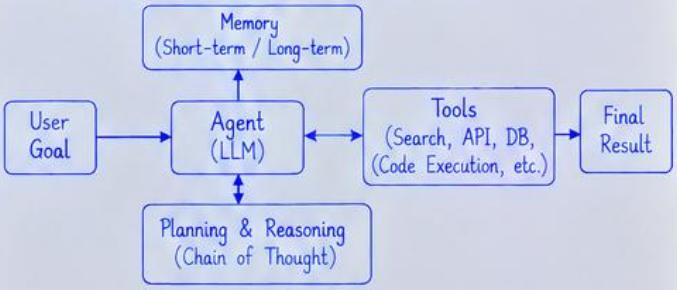
```
{
  "name": "get_weather",
  "arguments": {"location": "Delhi"}
}
```

Tool/Function
get_weather(location)

Result:
"The current temperature in Delhi is 28°C with clear skies."

17.8 Agents in NLP

- AI agents can plan, reason and use multiple tools to complete complex tasks.
- Components: Planning, Memory, Tool Use, Action, Observation, Reflection.
- Examples: AutoGPT, LangGraph, CrewAI.



17.9 Retrieval + Tools + Agents (Advanced RAG)

- Combines retrieval, tool use and agent reasoning.
- Agent first decides whether to use retrieval, call a tool, or generate directly.
- Used in enterprise knowledge assistants and research agents.



17.10 Key Takeaways

- Prompt engineering significantly improves LLM performance.
- Instruction tuning and RLHF make models more helpful and safe.
- Multimodal NLP enables text, image and audio understanding.
- Tool use and agents allow LLMs to perform real-world tasks.
- These advanced topics are essential for building production-ready NLP applications.

Rahul Yadav

18. Tools and Frameworks

18.1 Introduction

- NLP has a rich ecosystem of tools and frameworks that make it easier to build, train, fine-tune and deploy models.
- These tools provide pre-trained models, datasets, easy APIs and abstractions for common NLP tasks.
- Choosing the right tool depends on the use case, customization needs and deployment requirements.

18.2 Hugging Face Transformers

- Most popular open-source library for NLP models.
- Provides pre-trained models (BERT, GPT, T5, LLaMA, etc).
- Supports easy fine-tuning, inference and pipeline APIs.
- Works with PyTorch, TensorFlow and JAX.
- Large model hub with thousands of models.

Example: Using Hugging Face Pipeline

```
from transformers import pipeline
# Sentiment Analysis
nlp = pipeline("sentiment-analysis")
result = nlp("I love this product!")
print(result)
```

Output: `{{'label': 'POSITIVE', 'score': 0.9998}}`

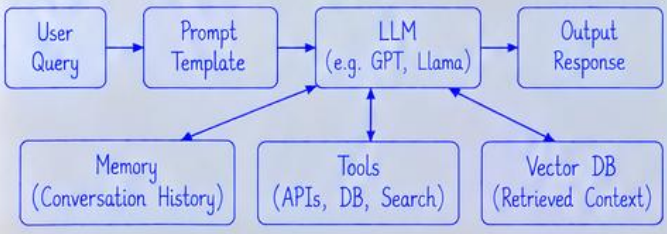
18.3 Popular Pre-trained Models in Hugging Face

Model	Use Case	Model Name (HF)
BERT	Text classification, NER	bert-base-uncased
RoBERTa	Better performance than BERT	roberta-base
DistilBERT	Lightweight, faster	distilbert-base-uncased
T5	Text-to-text (multi-task)	t5-base
GPT-2	Text generation	gpt2
LLaMA	Large language model	meta-llama/Llama-2-7b
BART	Summarization, generation	facebook/bart-large
DeBERTa	Improved BERT	microsoft/deberta-base

18.4 LangChain

- Framework to build LLM applications with tools, chains, memory and agents.
- Helps in connecting LLMs with external data sources (databases, APIs, documents, etc.).
- Provides abstractions for RAG, agents, conversation memory and tool usage.

LangChain Architecture (Simple)



18.5 LlamaIndex

- Framework for building LLM applications over your own data.
- Helps in data ingestion, indexing and retrieval.
- Supports multiple data sources (PDF, Docs, DB, APIs, etc.).
- Often used for RAG applications.

LlamaIndex Workflow



18.6 Vector Databases (Used with NLP)

- Store and search high-dimensional embeddings.
- Useful for semantic search, RAG and recommendation systems.

Vector DB	Type	Key Features	Use Case
FAISS	Open Source	Fast, in-memory similarity search	Research, small scale
Pinecone	Managed	Scalable, high availability	Production apps
Qdrant	Open Source	Filtering, metadata support	RAG, semantic search
Weaviate	Open Source	Graph + vector search	Enterprise use cases
Milvus	Open Source	Large scale, high performance	Big data applications

18.7 Other Useful Tools and Libraries

Tool / Library	Purpose	Use Case
spaCy	Industrial NLP library	Tokenization, NER, dependency parsing
NLTK	Traditional NLP library	Text preprocessing, classic algorithms
SentenceTransformers	Create sentence embeddings	Semantic similarity, retrieval
Gensim	Topic modeling	LDA, word embeddings
OpenAI API	Access GPT models	Text generation, chatbots
Gradio	Build ML web apps	Demo NLP models easily
Streamlit	Create interactive apps	NLP applications UI
FastAPI	Build REST APIs	Deploy NLP models
Docker	Containerize applications	Packaging and deployment
Kubernetes	Orchestrate containers	Scalability and production

18.8 Tool Selection Guide

Requirement	Recommended Tool(s)
Pre-trained models & fine-tuning	Hugging Face Transformers
Build LLM application with tools	LangChain
Work with your own documents (RAG)	LlamaIndex
Vector database for embeddings	FAISS / Pinecone / Qdrant
Traditional NLP tasks (tokenization, NER)	spaCy / NLTK
Create embeddings	SentenceTransformers
Build and deploy API	FastAPI + Docker + Kubernetes
Build demo UI	Streamlit / Gradio

18.9 Key Takeaways

- Use the right tool based on your project requirements.
- Hugging Face, LangChain and LlamaIndex are widely used in industry.
- Vector databases are critical for RAG and semantic search.
- Tools like FastAPI, Docker and Kubernetes help in production deployment.
- Combining multiple tools often gives the best results.

Rahul Yadav

19. Interview Questions and Final Revision

19.1 Common Interview Questions (Conceptual)

- What is NLP ?
- What are the main applications of NLP ?
- What is the difference between NLU and NLG ?
- What are the challenges in NLP ?
- What is a language model ?
- What is the purpose of embeddings ?
- How does attention mechanism work ?
- What are Transformers ?
- What is the difference between BERT, GPT and T5 ?
- What is fine-tuning ? When do we use it ?
- What is RAG and why is it useful ?
- How do you evaluate a text generation model ?
- What are the popular tools and frameworks for NLP ?

19.2 Important Differences

Topic	A	B
NLU vs NLG	NLU: Understands text	NLG: Generates text
BERT vs GPT	BERT: Bidirectional, better for understanding	GPT: Unidirectional, better for generation
Fine-tuning vs Prompting	Fine-tuning: Update model weights	Prompting: No weight update, just instructions
RAG vs Fine-tuning	RAG: Uses external knowledge	Fine-tuning: Learns from fixed data
Vector DB vs Traditional DB	Vector DB: Similarity search on embeddings	Traditional DB: Exact match search

19.3 Coding/Implementation Questions

- Implement text preprocessing (tokenization, stopwords, lemmatization).
- Generate word embeddings using a pre-trained model.
- Build a simple text classification model using BERT.
- Implement semantic search using sentence embeddings and a vector database (FAISS/Qdrant).
- Create a basic RAG pipeline using LangChain.
- Build a FastAPI endpoint to serve an NLP model.
- Write a function to calculate BLEU or ROUGE score.
- Implement inference using Hugging Face pipeline.

19.4 System Design Questions

- Design a scalable NLP system for a chatbot.
- How would you build a document Q&A system using RAG?
- Design a real-time translation service.
- How would you handle large document ingestion and indexing?
- How to scale an LLM application to millions of users?
- How would you monitor and manage model performance in production ?

19.5 Practical Project Ideas

Project	Description	Tools
Chat with PDF	Ask questions from PDF documents using RAG	LangChain, FAISS/ Qdrant, OpenAI
Text Classifier	Classify news articles or customer feedback	Hugging Face, BERT, FastAPI
Chatbot	Domain-specific chatbot	LangChain, GPT/Llama, Streamlit
Semantic Search	Search similar documents using embeddings	SentenceTransformers, Qdrant
Summarization App	Summarize long articles	T5/BART, Streamlit
Multimodal Q&A	Ask questions from images + text	LLaVA, LangChain

19.6 Tips for Interview Preparation

- Revise core NLP concepts (tokenization, embeddings, models).
- Practice implementing real projects.
- Understand pros and cons of different models and tools.
- Be ready to explain your past projects in detail.
- Solve coding problems related to NLP and text processing.
- Understand how to evaluate and improve model performance.
- Learn system design for real-world NLP applications.
- Stay updated with latest research and industry trends.

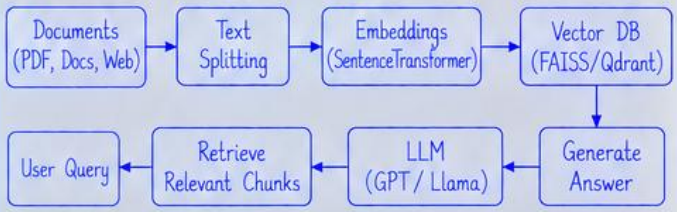
19.7 Quick Revision Cheat Sheet

Topic	Key Points
Text Preprocessing	Tokenization, normalization, stopwords, lemmatization
Text Representation	BoW, TF-IDF, Word2Vec, BERT embeddings
Language Models	N-gram, RNN, LSTM, Transformer, BERT, GPT
Attention	Self-attention, multi-head attention
Fine-tuning	Domain adaptation, PEFT, LoRA
RAG	Retrieve relevant docs + generate response
Evaluation Metrics	Accuracy, F1, BLEU, ROUGE, BERTScore
Deployment	FastAPI, Docker, Kubernetes, CI/CD
Tools	Hugging Face, LangChain, LlamaIndex, FAISS, Qdrant

19.8 Common MCQs (Examples)

1. Which model is bidirectional and used for understanding text ?
   a) GPT b) BERT c) T5 d) LLaMA
2. Which metric is commonly used for text summarization ?
   a) Accuracy b) BLEU c) F1 d) ROC-AUC
3. Which tool is used for building RAG applications ?
   a) OpenCV b) LangChain c) TensorFlow d) PyTorch
4. What does FAISS provide ?
   a) Text preprocessing b) Vector similarity search
   c) Model training d) Text generation
5. Which technique reduces model size ?
   a) Fine-tuning b) Quantization c) Prompting d) RAG

19.9 End-to-End NLP Project Flow (RAG Example)



19.10 Final Takeaways

- NLP is a rapidly growing field with many real-world applications.
- Understanding fundamentals + hands-on practice is key.
- Transformers, RAG and LLMs are the current state-of-the-art.
- Use the right tools and frameworks for faster development.
- Focus on building projects and solving practical problems.
- Be prepared for conceptual, coding and system design questions.
- Keep learning and stay updated with the latest advancements.

Rahul Yadav